

VectorLLM: Language-Controllable Autonomous Driving via Vectorized Scene Grounding

Abstract—Language-controlled autonomous driving requires an agent to precisely ground natural language instructions in map geometry and the motions of other road agents, and adapt its behavior when the scene or the instruction changes. Existing LLM-based driving models typically encode structured scene information via visual features, continuous embeddings, or text descriptions, introducing a representation gap as these input representations are fundamentally incompatible with the discrete token vocabulary natively processed by LLMs. To address this gap, we introduce VectorLLM, a framework that translates a continuous driving scene into discrete vector tokens and aligns them directly with an LLM’s discrete language token space. By expanding a pretrained decoder-only LLM with the discrete vector tokens, we fine-tune it to learn language, scene structure, and driving behavior together as a unified sequence modeling task. We evaluate VectorLLM on benchmarks built on the Waymo Open Motion Dataset (WOMD) and nuScenes. We show that VectorLLM achieves better driving question answering across WOMD-Reasoning and nuScenes-QA, demonstrating enhanced grounding. Furthermore, evaluations on our new human-annotated counterfactual datasets show that VectorLLM significantly outperforms baselines in its ability to adapt to changes in scenes and counterfactual language instructions. Crucially, representation ablations and attention analysis confirm that utilizing discrete scene tokens, rather than continuous alternatives, substantially improves grounding and controllability, providing an effective language interface for controllable autonomous driving.

I. INTRODUCTION

Natural language offers a flexible interface for human-vehicle interaction, enabling a vehicle to explain its behavior, answer questions about driving scenes, and adapt its maneuvers in response to human instructions. For such interaction to be effective, a driving model cannot just generate fluent sentences or predict future motion; it must accurately *ground* language in the driving scene and ensure its behavior is reliably *controllable* [1] through language. Recent driving LLM and VLM systems have explored this direction by representing driving scenes through 2D images or videos [2–5], object-level embeddings [6], or textual scene descriptions [7–10]. While these representations provide useful semantic context, they introduce a fundamental representation gap, as these formats are very different from the discrete tokens natively processed by LLMs. Consequently, precise grounding of road geometry, vehicle dynamics, and agent interactions remains challenging [11–14], as does reliably translating language instructions into driving behaviors [15, 16].

To bridge this representation gap, we introduce **VectorLLM**, a framework that provides an LLM with an explicit, token-level representation of structured driving scenes. Rather than relying on projected continuous features or

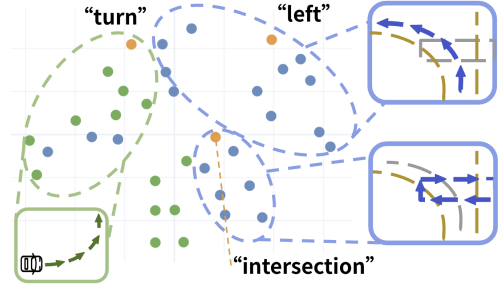


Fig. 1: **Language–vector-token alignment in the LLM latent space.** t-SNE visualization for a language-steering scene “turn left at the intersection.” Orange points are selected language tokens; blue and green points are vector-token occurrences from scene-relevant map geometry and the ego trajectory, selected before projection. Dashed regions are manually drawn to highlight rough local associations, not semantic clusters.

textual abstractions, VectorLLM converts map geometry and agent motions into discrete *vector tokens* and serializes them alongside natural language sentences in the LLM’s input sequence. This design makes geometric and motion information directly accessible to the language model, allowing the relationships among language, scene structure, and driving behavior to be learned jointly within a unified token space. By building upon the success of vectorized scene modeling for driving [17, 18] and discrete motion tokenization [19, 20], VectorLLM presents a new representation paradigm to encode driving scenes as a native LLM vocabulary.

Through supervised fine-tuning on driving question answering (QA) and trajectory generation, VectorLLM learns an aligned language and vector token representation (see Fig. 1). However, the model’s main advantage lies in its ability to generalize beyond these explicit training tasks. As shown in Fig. 2, we evaluate VectorLLM under counterfactual scene edits and language instructions without task-specific counterfactual training. This setup tests whether localized modifications to the scene or language elicit the correct changes in driving behavior. Furthermore, we qualitatively demonstrate the model’s ability in scene captioning, a task which is not included in training. Together, these tasks probe whether VectorLLM has learned reusable relationships among language, structured scene, and driving behavior rather than only reproducing the supervised driving patterns.

We evaluate VectorLLM on WOMD [21] and nuScenes [22] through driving QA and human-annotated counterfactual evaluations of scene edits and language steering. In addition, ablations on scene representations

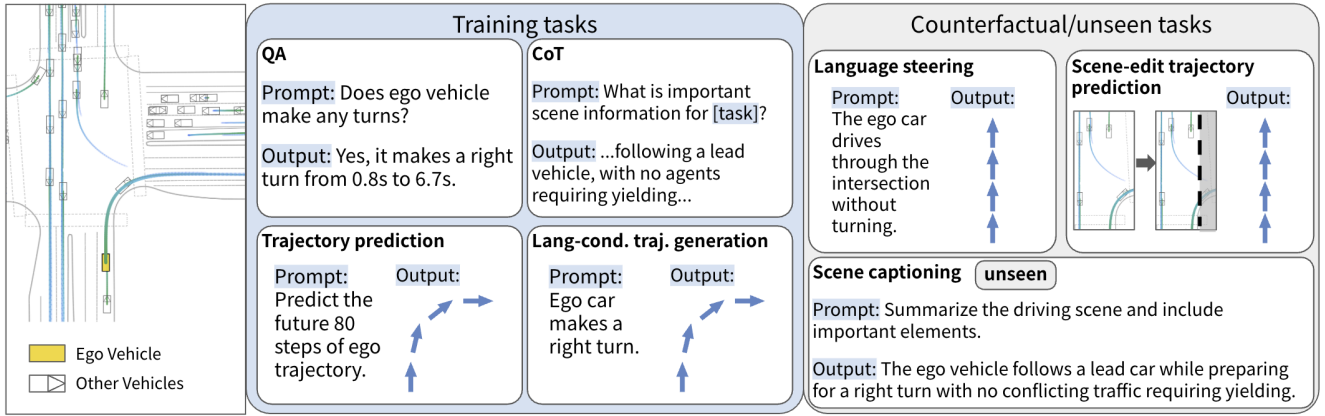


Fig. 2: **Overview of VectorLLM’s task capabilities.** Given a driving scene, VectorLLM performs the training tasks shown on the left and is further evaluated on counterfactual scene edits, language steering, and an unseen scene-captioning task shown on the right. The example prompts and outputs illustrate the different language and trajectory-generation capabilities evaluated in this work.

show that discrete vector tokens consistently outperform continuous alternatives. Attention and latent-space analyses reveal associations between language and relevant vector-token representations. In summary, this work makes the following contributions:

- We introduce **VectorLLM**, a novel framework that represents driving scenes as discrete vector tokens, enabling joint modeling of structured scenes and language within a unified LLM vocabulary.
- We demonstrate improved language grounding on driving QA across WOMB and nuScenes, showing that VectorLLM can retrieve relevant scene information to answer language queries.
- We introduce human-annotated counterfactual datasets to show VectorLLM’s controllability. Without specific counterfactual training, VectorLLM correctly adapts to changes in driving scenes and language instructions.
- Through representation ablations, attention and latent-space analyses, we validate that discrete tokenization is critical for bridging the representation gap to support grounding and controllability.

II. RELATED WORK

LLM/VLM-based Driving Recent works leverage LLMs and VLMs as planning or reasoning modules for autonomous driving. A common design encodes multi-view images into visual tokens [3–5, 23–25] and uses structured reasoning to convert visual scene understanding into meta-actions, explanations, or trajectories. However, precise driving-scene grounding remains challenging for visual-token representations, especially for 3D structure and agent interactions, motivating additional mechanisms for improving geometric reasoning [5, 12]. Alternatively, other methods abstract structured scene data into textual prompts [8–10]. While providing useful semantic context, text representation fundamentally compresses the scene and bottlenecks performance on manual prompt design. Recent driving vision-language-action (VLA) systems couple language-model reasoning with action or trajectory generation while representing the driving

scene through visual or projected continuous features [5, 12, 24, 25]. In contrast, VectorLLM avoids lossy textual compression and continuous feature mismatch by directly converting vectorized scenes into discrete vector tokens so they can be jointly learned with language within the LLM’s native token sequence.

Structured Driving Scene Representations Explicitly modeling driving scenes with vectorized [17, 18] and interaction- or graph-based [26–28] representations has been effective for trajectory forecasting and planning. Recent works further show that continuous driving scenes can be discretized into token sequences for traffic modeling and motion generation [19, 20]. VectorLLM builds on these paradigms to extend the utility of discrete tokens by integrating them natively with natural language. By converting driving scenes into discrete vector tokens and serializing them alongside text within the LLM’s context window, VectorLLM jointly learns the relationships among language, scene structure, and driving behavior within a unified sequence.

Language Grounding and Control in Driving Effective integration of natural language in autonomous driving requires grounding language in relevant scene elements and generating behavior that follows language instructions reliably. Recent driving QA benchmarks [11, 14, 24, 29] highlight challenges in spatial, interaction, temporal, and counterfactual reasoning, while prior control-oriented works use diffusion-guided constraints [30], text-based scenario editing [31], or instruction-paired trajectories [32–35]. Among these methods, iMotion-LLM [35] also combines vectorized driving scenes with language for instruction-conditioned trajectory generation. However, it relies on a GameFormer-based encoder [36] to encode the vectorized scene and projects the resulting continuous scene features into the LLM. VectorLLM eliminates this continuous projection and represents scenes directly as discrete vector tokens. Furthermore, when generating trajectories, iMotion-LLM maps the LLM latent output back into the GameFormer feature space and uses its trajectory decoder to produce the final trajectory, whereas VectorLLM does not rely on an external decoder,

generating trajectories using the discrete vector tokens.

III. PROBLEM FORMULATION

We formulate language-controlled autonomous driving as a unified discrete sequence generation problem. Let S denote the driving scene, comprising local map geometry, the ego vehicle’s motion history, and surrounding agents’ motion histories. Let L_{in} denote the natural language input which may specify a driving instruction, a question, or a task prompt. By mapping the driving scene S and language input L_{in} into a shared discrete token vocabulary, the model generates a task-dependent output sequence O , corresponding to either a linguistic response O_{lang} or a planned future trajectory O_{traj} . We define the learning problem as modeling the conditional probability:

$$p_{\theta}(O | S, L_{\text{in}}), \quad O \in \{O_{\text{lang}}, O_{\text{traj}}\}. \quad (1)$$

To enhance the performance and encourage interpretable decisions, the output sequence O may include an intermediate chain-of-thought (CoT) reasoning trace, preceding the final task response. Specifically, for language tasks, O_{lang} yields the textual reasoning trace followed by the language response; for trajectory generation, O_{traj} emits the reasoning about lane or goal selections followed by the sequence of discrete tokens representing the future ego trajectory.

To achieve this unified sequence modeling while minimally modifying the base LLM, we aim to expand the LLM vocabulary but keep the network unchanged. Let $\mathcal{V}_l$ denote the original language vocabulary and $\mathcal{V}_s$ denote the scene vocabulary used to discretize driving scenes. We defined the expanded model vocabulary $\mathcal{V}$ and construct the LLM input context $\mathbf{x}$ as:

$$\mathcal{V} = \mathcal{V}_l \cup \mathcal{V}_s, \quad \mathbf{x} = \mathbf{s} \parallel \mathbf{l}, \quad (2)$$

where $\mathbf{s}$ is a sequence of vector tokens drawn from $\mathcal{V}_s$ that encodes the driving scene S , $\mathbf{l}$ is the sequence of language tokens drawn from $\mathcal{V}_l$ corresponding to L_{in} , and $\parallel$ denotes sequence concatenation. Thus, $\mathbf{x}$ combines scene and language information into a single input sequence over $\mathcal{V}$ for the LLM. Under this formulation, language grounding is reflected as the model’s capacity to accurately associate language tokens in $\mathbf{l}$ with the scene information in $\mathbf{s}$, while controllability is achieved when causal perturbations to either the scene $\mathbf{s}$ or the instruction $\mathbf{l}$ elicit corresponding changes in the generated output sequence.

IV. METHOD

This section details the design of VectorLLM, focusing on how driving scenes are discretized into tokens to enable joint learning of language, scene structure, and driving behavior. Figure 3 shows the overall architecture of VectorLLM. A driving scene is first vectorized and then discretized into a series of tokens, called *vector tokens*. These vector tokens are concatenated with language tokens to construct the LLM’s input context. The VectorLLM employs a pretrained decoder-only LLM as the backbone. We use a bidirectional attention

mask for trajectory output to account for the temporal structure of trajectory reasoning while maintaining standard causal attention elsewhere. We train the model with chain-of-thought reasoning. The reasoning trace explicitly selects and references relevant scene elements using vector tokens, providing intermediate supervision that relates language generation with evidence in the driving scene. Finally, the generated output tokens are decoded into language answers, intermediate driving goals, or trajectory vectors depending on the conditioned task.

A. Vectorization and Tokenization of Driving Scenes

Driving Scene Vectorization VectorLLM takes vectorized driving-scene elements as input. The static map context is provided by HD-map elements.

This input scene comprises the coordinates for the static map elements (e.g., lanes, road edges, crosswalk, and stop signs) and the dynamic agent motions (including the trajectory histories of the ego and surrounding agents). To ensure alignment across scenes, all coordinates are transformed into the normalized frame of the ego agent. The key step is to express both map elements and agent motions within the same vectorization scheme: an ordered sequence of two-dimensional displacement vectors. For an agent trajectory with positions $\{p_t\}_{t=1}^T$, each raw motion vector is $v_t = p_t - p_{t-1} = (\Delta x_t, \Delta y_t)$. For static map elements, the same operation is applied spatially along the road graph structure: map polylines are re-sampled into ordered points using curvature-adaptive step sizes: shorter vectors are used in high-curvature regions and longer vectors in near-linear regions, with resolutions of 0.5m, 1.0m, 1.5m, and 2.0m. Each raw road vector then represents the spatial displacement $(\Delta x, \Delta y)$ between adjacent sampled points.

Tokenization These continuous vectors are discretized into tokens. To build this vocabulary, we collect per-step agent displacement vectors from the WOMD training split and apply k-disks clustering [19] to extract representative centroids. Each centroid is assigned one special vector token, denoted VEC_i . This creates a scene vocabulary $\mathcal{V}_s$ that we use to represent both temporal agent motions and static map structures. Since a vector token only encodes local displacement, it lacks driving semantics such as whether a token sequence belongs to the ego vehicle or a lane. To provide this context, we introduce high-level semantic marker tokens and delimiter tokens that identify the semantic type and boundaries of map, ego, and surrounding-agent token sequences. During tokenization, since naive nearest-centroid assignment can accumulate reconstruction drift over long road polylines, we employ a beam-search assignment procedure that selects a sequence of vector tokens to minimize the cumulative displacement error of the reconstructed polyline. We empirically validate that this procedure mitigates cumulative drift, reconstructing road and trajectory polylines with an error below 0.03 m in WOMD.

LLM Input Context After tokenization, the driving scene is serialized as a single token sequence to combine with

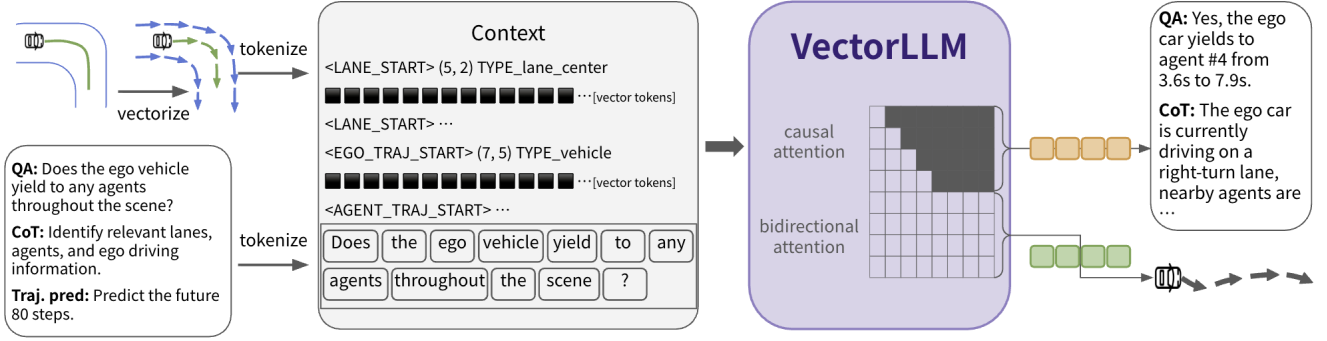


Fig. 3: **Overview of VectorLLM.** (1) VectorLLM tokenizes vectorized map geometry, ego- and other-agent motions, and language prompts into one LLM context. (2) A decoder-only LLM processes this shared sequence, with CoT supervision providing scene-grounded intermediate reasoning. (3) The same model produces language answers, control-relevant goals, and trajectory vectors.

language prompts. A representative context structure includes these blocks:

[MAP, ROAD_0, ROAD_1, ..., EGO_TRAJ, AGENT_TRAJ_1, AGENT_TRAJ_2, ..., q, TRJ_MARKER]

where q is an optional language question, instruction, or auxiliary prompt. The actual map and agent blocks contain their semantic marker tokens and delimiter tokens when available, and the corresponding VEC sequences. The map blocks typically occupy the largest fraction of the context because road geometry contains many spatial vectors. During training, scenes longer than the N_e -token training limit are dropped. At inference time, this training-time filter is not applied.

B. Model Architecture

LLM Backbone VectorLLM builds upon a pretrained decoder-only LLM by expanding its vocabulary to incorporate vector tokens. Rather than repurposing existing tokens, we preserve the original language vocabulary and expand both the input embedding matrix and the output language-modeling head to include the new vector and special tokens. These newly added embeddings are initialized using the mean and covariance of the pretrained embedding distribution, while existing language-token embeddings are not overwritten. Our model architecture relies purely on the LLM backbone. There is no need for a separate map encoder, motion encoder, or intermediate projection layer. VectorLLM is trained by full fine-tuning an LLM: the pretrained transformer parameters, original language token embeddings, newly added token embeddings, and expanded output head are all trainable.

Chain-of-thought (CoT) Reasoning VectorLLM employs CoT reasoning to explicitly align language with vector tokens before committing to a task output. The CoT traces follow a driving-oriented reasoning flow: “identify the ego motion state $\rightarrow$ select relevant map context and nearby agents $\rightarrow$ reason about interactions or feasible maneuvers $\rightarrow$ support the final answer or trajectory generation.” For QA, we construct the trace to focus only on evidence relevant to the question. For trajectory generation, we construct the trace using a broader scope of future driving options, e.g., turning

or yielding, to explain the intended maneuver. To improve reliability for long-horizon planning, all trajectory generation tasks in VectorLLM utilize goal-conditioned generation similar to [37, 38]. Following the CoT rationale, the model first predicts three sparse future waypoints to represent ego’s goal. This goal is not decoded as a discrete token sequence; instead, a lightweight MLP head reads the LLM hidden states at designated goal positions to regress continuous metric coordinates. The vector token trajectory is then generated conditioned on this intermediate goal.

Decoding Strategies VectorLLM uses different decoding strategies for language and trajectory generation. For language outputs, we preserve the standard autoregressive decoding with causal attention. Trajectory outputs are generated using parallel decoding similar to OpenVLA-OFT [39]. The designated goal and trajectory output positions are initialized with zero embeddings in the LLM input context. Bidirectional attention is applied only to these target positions. Under this masking scheme, target positions can attend to all scene and prompt tokens in the context, and they can also attend to every other target position. Context tokens use causal attention and are prevented from attending to target positions. This design gives one-way information flow from the serialized scene context to the target positions while allowing all future motion positions to be decoded jointly in a single pass. Finally, the expanded language-modeling head produces a distribution over the vector tokens for each target position. The selected tokens are mapped back to their displacement centroids and reconstructed into metric coordinates by cumulative summation.

C. Training Pipeline

VectorLLM is trained in two stages, with all model parameters remaining trainable in both stages. In Stage I, we perform vector-token pretraining using standard next-token prediction over serialized driving scenes. This stage exposes the pretrained LLM to map and motion structures as ordinary token sequences. In Stage II, we perform multi-task fine-tuning on driving question answering, goal prediction, and trajectory generation. The training objective combines the losses active for each task. Vector-token pretraining and

language outputs use standard autoregressive cross-entropy over the expanded vocabulary. Trajectory generation uses cross-entropy over the vector tokens at each trajectory target position. The intermediate goal positions are supervised using an ℓ_2 loss applied to the MLP goal head.

V. EXPERIMENTS

We conduct extensive experiments on WOMD and nuScenes to address these research questions.

- RQ1** How effectively does the shared token space ground language in driving scenes?
- RQ2** Does language-scene grounding translate to controllable driving behavior?
- RQ3** What enables VectorLLM to align language, scene structure, and driving behavior?

A. Experimental Setup

Implementation We use Qwen3-0.6B [40] as the pre-trained LLM backbone, though VectorLLM can be applied to other decoder-only LLMs. We set $|\mathcal{V}_s| = 1024$, where reconstruction error drops from about 0.10 m at 384 tokens and 0.05 m at 512 tokens to 0.03 m, with little improvement at 2048 or 4096 tokens. During training, we limit the input context to $N_c = 12,288$ tokens; dropping scenes that exceed this length removes just 0.6% of the training scenes from WOMD. We use AdamW with a 10^{-5} learning rate, 10% warmup, and cosine decay. QA training uses 116,899 examples for 3 epochs with an effective batch size of 32, taking 30 hours on four A100; trajectory generation uses 50,827 examples for 8 epochs with an effective batch size of 48, taking 74 hours on six RTX A4500.

Map Inputs Standard VectorLLM uses polylines from HD-map as input. To validate VectorLLM’s compatibility with perception inputs, we additionally run a variant to use the vectorized map elements predicted from multiview images by a pretrained MapTR model [41]. We include this variant across Tables II–IV to establish a performance lower-bound when integrating VectorLLM with a perception module. Notably, despite not consuming dense visual features, the VectorLLM+MapTR variant can match the grounding performance of existing VLA and significantly outperform SoTA models in controllability.

Training Data We utilize the WOMD training split as the primary data source for vector-token pretraining and multi-task fine-tuning. WOMD provides rich map elements, agent motion histories, and future trajectories in a format that can be vectorized and tokenized to VectorLLM’s inputs. To construct multi-task fine-tuning data, we first apply rule-based filters to extract task-relevant map, motion, and interaction facts from each vectorized scene. We then prompt Qwen3-4B-Instruct-2507 to express the extracted facts as task-specific natural language descriptions for QA, CoT, and language-conditioned trajectory generation. These generated training examples provide QA targets and driving-oriented CoT supervision grounded in evidence from the serialized scene. Before evaluation, VectorLLM is fine-tuned on the

designated training data for each benchmark. The human-annotated counterfactual datasets, comprising scene edits and language steering, are held out for evaluation and are not used for training. This ensures the evaluation of model’s controllability is intrinsic rather than a product of task-specific counterfactual training.

Evaluation Benchmarks To assess language grounding, we evaluate driving-specific question answering on the established WOMD-Reasoning [29] and nuScenes-QA [11] benchmarks. To evaluate controllability, we introduce two human-annotated, evaluation-only counterfactual benchmarks. The WOMD scene-edit benchmark contains 300 validation scenes with edits to lane geometry, surrounding-agent presence or behavior, or ego motion history, paired with target ego waypoints that reflect reasonable responses to the edited conditions. The nuScenes language-steering benchmark contains 200 test scenes in which human annotators specify alternative feasible driving instructions with their corresponding target waypoints. Both benchmarks were annotated by the same 15 annotators, with quality controlled through visual inspection and automated checks for implausible sharp turns, acceleration, and deceleration.

B. Language Grounding (RQ1)

We evaluate VectorLLM’s language-grounding capability through question answering. Answering a scene-dependent question requires the model to identify and use the map, motion, or interaction information relevant to the language query, making QA a direct assessment of whether language is grounded in the structured driving scene.

WOMD-Reasoning Table I compares VectorLLM against Motion-LLaVA, a motion-language baseline from Li et al. [29]. VectorLLM improves over Motion-LLaVA on both accuracy within a 1.0-meter tolerance ($\text{Acc}_{1.0}$) and language metrics (ROUGE). These results provide evidence that VectorLLM can ground language queries in the structured scene information needed for driving QA.

TABLE I: QA performance on WOMD-Reasoning.

Method	Modality	$\text{Acc}_{1.0}\uparrow$	ROUGE $\uparrow$
Motion-LLaVA [29]	HD map	56.2%	0.792
VectorLLM	HD map	61.0%	0.804

nuScenes-QA Table II summarizes the results on nuScenes-QA, which evaluates language grounding using question-answer pairs constructed from the nuScenes scene graph. VectorLLM achieves the highest H0, H1, and overall accuracy. Its performance on both zero-hop and one-hop questions shows that the model can use information in the tokenized driving scenes for direct scene queries (H0) and relational questions (H1), extending the grounding evidence from WOMD-Reasoning to nuScenes. Notably, OpenDriveVLA uses rich visual features directly, whereas VectorLLM+MapTR uses processed vision input through MapTR; despite this difference, VectorLLM+MapTR performs on par with OpenDriveVLA [12].

TABLE II: Performance on nuScenes-QA, split by zero-hop (H0) and one-hop (H1) questions.

Method	Modality	H0↑	H1↑	Acc.↑
LLaMA-AdapV2 [42]	Vision	15.1	4.8	9.6
LLaVA1.5 [43]	Vision	25.7	41.5	26.2
OpenDriveVLA-0.5B [12]	Vision	62.3	56.5	58.4
VectorLLM	HD map	69.9	60.4	63.5
VectorLLM+MapTR	Vision (MapTR)	61.7	55.9	57.8

C. Controllability of Driving Behavior (RQ2)

We evaluate controllability in the two newly collected counterfactual datasets introduced in Sec. V-A. Scene edits test whether the generated behavior responds to changes in the driving scene, while language steering keeps the scene fixed and changes the requested driving instruction. For each input condition, the evaluated model generates six trajectory hypotheses. We report open-loop best-of-six target reachability, defining a successful response when at least one generated hypothesis falls within 1.0m radius of the annotated target waypoint at the annotated horizon. This metric tests whether the intended counterfactual response is captured within the model’s generated trajectory distribution.

Counterfactual Scene Edits We evaluate performance on the WOMD counterfactual scene-edit benchmark against established motion forecasting models, MTR [38] and M2I [28]. As shown in Table III, VectorLLM and VectorLLM+MapTR achieve the highest overall target reachability (64.7% and 60.5%, respectively), outperforming 53.3% for MTR, the stronger baseline. Furthermore, VectorLLM achieves higher target reachability than both baselines across every edit type. This consistent advantage across lane, agent, and ego-history edits shows that VectorLLM adapts its generated behavior to changes in map geometry, surrounding agents, and the ego vehicle’s recent motion. VectorLLM+MapTR retains similar behavior with processed vision input, showing that responsiveness to scene changes does not depend on annotated HD-map input.

TABLE III: Counterfactual target reachability (1.0m) for scene edits, split by edit type on WOMD.

Method	Modality	Lane↑	Agent↑	Ego History↑	All↑
MTR [38]	HD map	47.0%	52.0%	61.0%	53.3%
M2I [28]	HD map	39.0%	35.0%	18.0%	30.7%
VectorLLM	HD map	65.0%	60.0%	69.0%	64.7%
VectorLLM+MapTR	Vision (MapTR)	61.5%	57.0%	63.0%	60.5%

Counterfactual Language Instructions We evaluate language-steering performance from nuScenes against CTG++ [30], a diffusion-based generation model with language guidance, and iMotion-LLM [35], an LLM baseline using vectorized scene inputs. As shown in Table IV, VectorLLM achieves 76.0% average target reachability, outperforming the strongest baseline iMotion-LLM at 68.3%. VectorLLM+MapTR retains 71.8% average target reachability with processed vision input, showing that the language-steering capability also persists without annotated HD-map input. These results show that VectorLLM more reliably adapts its generated trajectories to the requested alternative language instructions. Without the language instruction, average target reachability drops to 29.8%, showing that the language

prompts materially redirect the generated behavior rather than merely defaulting the scene-conditioned future.

TABLE IV: Counterfactual target reachability (1.0m) for language-steering at different horizons on nuScenes.

Method	Modality	w/ Lang.	Avg.↑	@1s↑	@2s↑	@3s↑
VectorLLM	HD map	✗	29.8%	49.0%	30.5%	10.0%
CTG++ [30]	HD map	✓	15.0%	31.5%	11.0%	2.5%
iMotion-LLM [35]	HD map	✓	68.3%	80.0%	71.5%	53.5%
VectorLLM	HD map	✓	76.0%	93.5%	80.0%	64.5%
VectorLLM+MapTR	Vision (MapTR)	✓	71.8%	89.0%	78.5%	58.0%

D. Language and Vector Token Representation Alignment (RQ3)

To understand whether and how VectorLLM’s discrete vector-token representation contributes to grounding and controllability beyond the structured scene data provided to the model, we conduct evaluation through representation ablation, attention and latent-space analyses, and inference-time CoT reasoning ablation.

Effect of Scene Representation Table V compares VectorLLM against two continuous alternatives across the grounding and controllability evaluations. First, the “GameFormer + Qwen” baseline receives the same HD-map and agent information as VectorLLM, but encodes the scene as continuous features rather than discrete vector tokens. GameFormer serves only as the scene encoder, without its trajectory decoder; the Qwen backbone, supervision, training data, and evaluation procedure are otherwise unchanged. VectorLLM improves QA from 51.3% to 61.0%, scene-edit controllability from 54.0% to 64.7%, and language-steering controllability from 46.3% to 76.0%. These gains show that the improvement is not explained merely by access to structured HD-map and agent information, but also depends on how that information is represented to the LLM.

To more directly isolate the discrete token representation, we use a “Token Encoder + Qwen” baseline. Both methods start from the same vectorized and tokenized scene, but the Token Encoder converts the token sequence into continuous embeddings before passing it to the LLM, while VectorLLM retains discrete vector tokens. The Token Encoder is trained to align with embeddings from the OpenDriveVLA vision encoder, with the downstream Qwen backbone, supervision, training data, and evaluation procedure unchanged. It reaches 47.6% on QA, 39.3% on scene edits, and 50.7% on language steering, compared with VectorLLM’s 61.0%, 64.7%, and 76.0%. Because the scene input is identical, this comparison more directly attributes the gains to discrete vector tokens rather than continuous embeddings.

TABLE V: Scene-representation ablations. QA is WOMD-Reasoning Acc_{1.0}; scene edit and language steering are average best-of-six target-reachability scores.

Method	Modality	QA↑	Scene edit↑	Lang. steering↑
VectorLLM	HD map	61.0%	64.7%	76.0%
GameFormer + Qwen	HD map	51.3%	54.0%	46.3%
Token Encoder + Qwen	HD map	47.6%	39.3%	50.7%

Token Attention Visualizations To understand how VectorLLM grounds language and vector tokens, we visualize

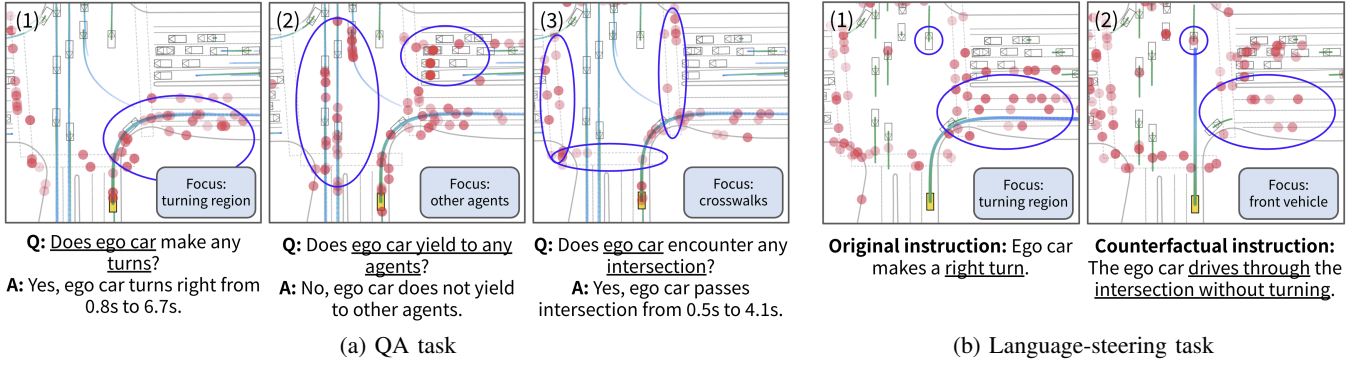


Fig. 4: **Token attention visualization.** Red dots indicate high-attention vector tokens, with darker dots corresponding to larger attention weights. Blue circles are manually added by the authors to highlight scene regions receiving notable model attention. The ego vehicle is yellow.

token attention in Fig. 4. For each example, we extract attention from the underlined language tokens to serialized vector-token occurrences, average the weights across the last 10 LLM layers, and map the highest-attended occurrences back to the scene. Red dots show these attended locations, with darker dots indicating larger attention weights.

In Fig. 4(a), “turn” attends primarily to the turning road geometry and ego path, “yield” attends to relevant surrounding agents, and “intersection” attends to the intersection and nearby road structure. Fig. 4(b) keeps the scene fixed while changing the instruction; attention shifts from the turning lane and surrounding road geometry toward the intersection approach and nearby traffic context. These examples provide qualitative evidence that the language context changes which parts of the vectorized scene are emphasized by the model.

We next examine whether attention adapts to a scene edit while the language context is fixed. In Fig. 5(1), the ego approaches and stops before the intersection, with attention distributed over surrounding stopped agents. In Fig. 5(2), an annotator adds a vehicle that travels in the ego lane and decelerates to a stop directly ahead of the ego, causing attention to shift toward the newly introduced vehicle. Together with Fig. 4(b), this provides qualitative evidence that the model reallocates attention in response to changes in both language and scene context.

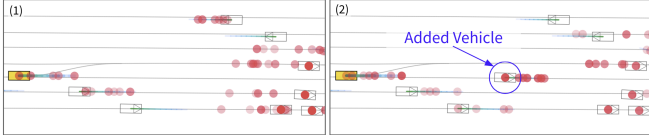


Fig. 5: **Attention under a counterfactual scene edit: adding an agent.** (1) Original scene. (2) Agent-edited scene, where the annotator-added vehicle is circled in blue.

Token Alignment in Latent Space We use t-SNE to examine whether language tokens and vector tokens with related driving semantics develop nearby contextual representations. In Fig. 1, we extract hidden states of selected language tokens and vector tokens associated with the map geometry or generated trajectories, average their representations over LLM layers 18–24 while excluding the final output layer, and project them into two dimensions. Words about

maneuvers such as “turn” and “left” lie near vector tokens associated with the corresponding turning motion, while “intersection” lies near vector tokens associated with relevant map geometry. These local relationships provide qualitative evidence that the model learns semantic associations between language and vector token representations of scene structure and driving behavior.

Inference-Time CoT Ablation An effective token representation should enable the LLM to reason over token sequences. To verify this, we use the same trained VectorLLM to evaluate three inference settings: normal CoT, corrupted CoT, and no CoT. Corrupted CoT preserves the format and approximate length of the intermediate reasoning but replaces its content with inconsistent or irrelevant information, while no CoT skips the reasoning stage and produces the task output directly. As shown in Table VI, full CoT performs best on QA and both controllability evaluations, followed by corrupted CoT and then no CoT. This shows that both the presence of intermediate reasoning and the consistency of its content contribute to performance.

TABLE VI: Inference-time CoT ablation on WOMD-Reasoning QA and controllability.

Setting	QA Acc _{1.0} ↑	ROUGE↑	Lang. steer↑	Scene edit↑
VectorLLM w/ full CoT	61.0%	0.804	76.0%	64.7%
w/ corrupted CoT	53.7%	0.715	70.3%	60.7%
w/o CoT	51.2%	0.712	66.7%	58.0%

Backbone Scaling We evaluate VectorLLM with Qwen3 and Gemma-3 backbones of different sizes on the language steering task. As shown in Fig. 6, performance improves with backbone size across both model families, showing that the result is not specific to the default Qwen3-0.6B backbone.

VI. CONCLUSION

We present VectorLLM, a novel framework that represents map geometry and agent motion as discrete vector tokens, enabling structured driving scenes to be jointly modeled with natural language within a unified sequence. Across driving QA and counterfactual evaluations, we demonstrate that this native discrete formulation successfully bridges the representation gap. It supports grounding and enables the model to adapt its generated trajectories in response to scene edits and alternative instructions. Our approach

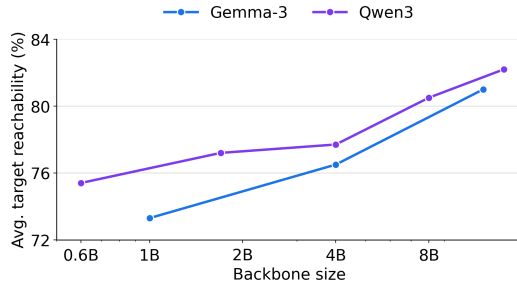


Fig. 6: Performance scaling of the language steering task across backbone sizes on Gemma-3 and Qwen3.

currently depends on the quality of structured scene inputs and evaluates controllability in open-loop settings. Future work includes integrating raw perception more tightly with this token representation and extending the framework to closed-loop evaluation. We hope this work opens up a path for leveraging discrete token representations to effectively integrate driving scenes with language modeling to enable more controllable language-based autonomous driving.

REFERENCES

- [1] S. Gao *et al.*, “Vista: A generalizable driving world model with high fidelity and versatile controllability,” *Advances in Neural Information Processing Systems*, 2024.
- [2] Wayve, “LINGO-1: Exploring natural language for autonomous driving,” <https://wayve.ai/thinking/lingo-natural-language-autonomous-driving>, 2023.
- [3] X. Tian *et al.*, “DriveVLM: The convergence of autonomous driving and large vision-language models,” in *Proceedings of The 8th Conference on Robot Learning*. PMLR, 2024.
- [4] Z. Xu *et al.*, “DriveGPT4: Interpretable end-to-end autonomous driving via large language model,” *IEEE Robotics and Automation Letters*, 2024.
- [5] J.-J. Hwang *et al.*, “EMMA: End-to-end multimodal model for autonomous driving,” *Transactions on Machine Learning Research*, 2025.
- [6] L. Chen *et al.*, “Driving with LLMs: Fusing object-level vector modality for explainable autonomous driving,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024.
- [7] J. Mao *et al.*, “GPT-Driver: Learning to drive with GPT,” *arXiv preprint arXiv:2310.01415*, 2023.
- [8] H. Sha *et al.*, “LanguageMPC: Large language models as decision makers for autonomous driving,” *arXiv preprint arXiv:2310.03026*, 2023.
- [9] D. Fu *et al.*, “Drive like a human: Rethinking autonomous driving with large language models,” in *Winter Conference on Applications of Computer Vision Workshops (WACVW)*. IEEE, 2024.
- [10] L. Wen *et al.*, “DiLu: A knowledge-driven approach to autonomous driving with large language models,” in *International Conference on Learning Representations*, 2024.
- [11] T. Qian *et al.*, “nuScenes-QA: A multi-modal visual question answering benchmark for autonomous driving scenario,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.
- [12] X. Zhou *et al.*, “OpenDriveVLA: Towards end-to-end autonomous driving with large vision language action model,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.
- [13] Y. Lin, S. Illing, and M. Althoff, “Sandra: Safe large-language-model-based decision making for automated vehicles using reachability analysis,” *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026.
- [14] C. Fruhwirth-Reisinger *et al.*, “STSBench: A spatio-temporal scenario benchmark for multi-modal large language models in autonomous driving,” in *The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2025.
- [15] Y. Ma *et al.*, “dVLM-AD: Enhance diffusion vision-language-model for driving via controllable reasoning,” *arXiv preprint arXiv:2512.04459*, 2025.
- [16] F. Schmidt *et al.*, “LAD-Drive: Bridging language and trajectory with action-aware diffusion transformers,” *arXiv preprint arXiv:2603.02035*, 2026.
- [17] J. Gao *et al.*, “VectorNet: Encoding HD maps and agent dynamics from vectorized representation,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020.
- [18] B. Jiang *et al.*, “VAD: Vectorized scene representation for efficient autonomous driving,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.
- [19] J. Phillion, X. B. Peng, and S. Fidler, “Trajenglish: Traffic modeling as next-token prediction,” in *The Twelfth International Conference on Learning Representations*, 2024.
- [20] W. Wu *et al.*, “SMART: Scalable multi-agent real-time motion generation via next-token prediction,” *Advances in Neural Information Processing Systems*, 2024.
- [21] S. Ettinger *et al.*, “Large scale interactive motion forecasting for autonomous driving: The Waymo Open Motion Dataset,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021.
- [22] H. Caesar *et al.*, “nuScenes: A multimodal dataset for autonomous driving,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.
- [23] H. Shao *et al.*, “LMDrive: Closed-loop end-to-end driving with large language models,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.
- [24] S. Wang *et al.*, “OmniDrive: A holistic vision-language dataset for autonomous driving with counterfactual reasoning,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025.
- [25] K. Renz, L. Chen, E. Arani, and O. Sinavski, “SimLingo: Vision-only closed-loop autonomous driving with language-action alignment,” in *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [26] M. Liang *et al.*, “Learning lane graph representations for motion forecasting,” in *ECCV*, 2020.
- [27] Z. Zhou *et al.*, “HiVT: Hierarchical vector transformer for multi-agent motion prediction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [28] Q. Sun *et al.*, “M2I: From factored marginal trajectory prediction to interactive prediction,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [29] Y. Li *et al.*, “WOMD-Reasoning: A large-scale dataset for interaction reasoning in driving,” in *Proceedings of the 42nd International Conference on Machine Learning*, 2025.
- [30] Z. Zhong *et al.*, “Language-guided traffic simulation via scene-level diffusion,” in *Conference on Robot Learning*, 2023.
- [31] S. Tan *et al.*, “Language conditioned traffic generation,” in *7th Annual Conference on Robot Learning*, 2023.
- [32] W.-J. Chang *et al.*, “LANGTRAJ: Diffusion model and dataset for language-conditioned trajectory simulation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2025.
- [33] S. Tan *et al.*, “Promptable closed-loop traffic simulation,” in *8th Annual Conference on Robot Learning*, 2024.
- [34] M. Peng *et al.*, “LC-LLM: Explainable lane-change intention and trajectory predictions with large language models,” *arXiv preprint arXiv:2403.18344*, 2024.
- [35] A. Felemban *et al.*, “iMotion-LLM: Instruction-conditioned trajectory generation,” in *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2026.
- [36] Z. Huang, H. Liu, and C. Lv, “GameFormer: Game-theoretic modeling and learning of transformer-based interactive prediction and planning for autonomous driving,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [37] J. Gu, C. Sun, and H. Zhao, “DenseTNT: End-to-end trajectory prediction from dense goal sets,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [38] S. Shi *et al.*, “Motion transformer with global intention localization and local movement refinement,” in *Advances in Neural Information Processing Systems*, 2022.
- [39] M. J. Kim, C. Finn, and P. Liang, “Fine-Tuning Vision-Language-Action Models: Optimizing Speed and Success,” in *Proceedings of Robotics: Science and Systems*, 2025.
- [40] A. Yang *et al.*, “Qwen3 technical report,” *arXiv preprint arXiv:2505.09388*, 2025.
- [41] B. Liao *et al.*, “MapTR: Structured modeling and learning for online vectorized HD map construction,” in *International Conference on Learning Representations*, 2023.
- [42] P. Gao *et al.*, “LLaMA-Adapter V2: Parameter-efficient visual instruction model,” *arXiv preprint arXiv:2304.15010*, 2023.
- [43] H. Liu *et al.*, “Improved baselines with visual instruction tuning,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.